LLM’s

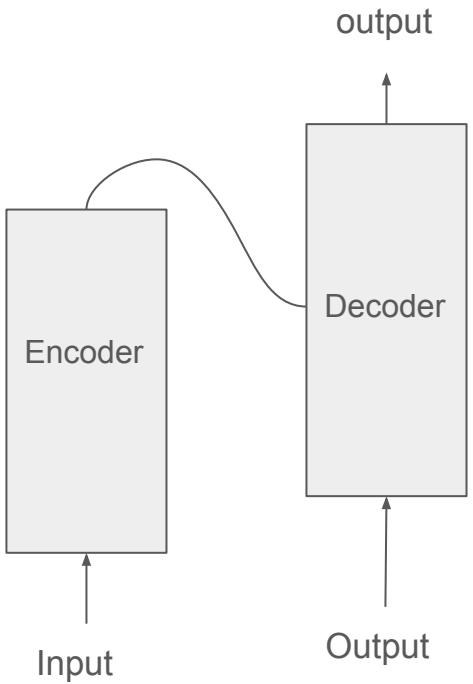
1-Background

2-LLM’s Structure

3-Optimizations

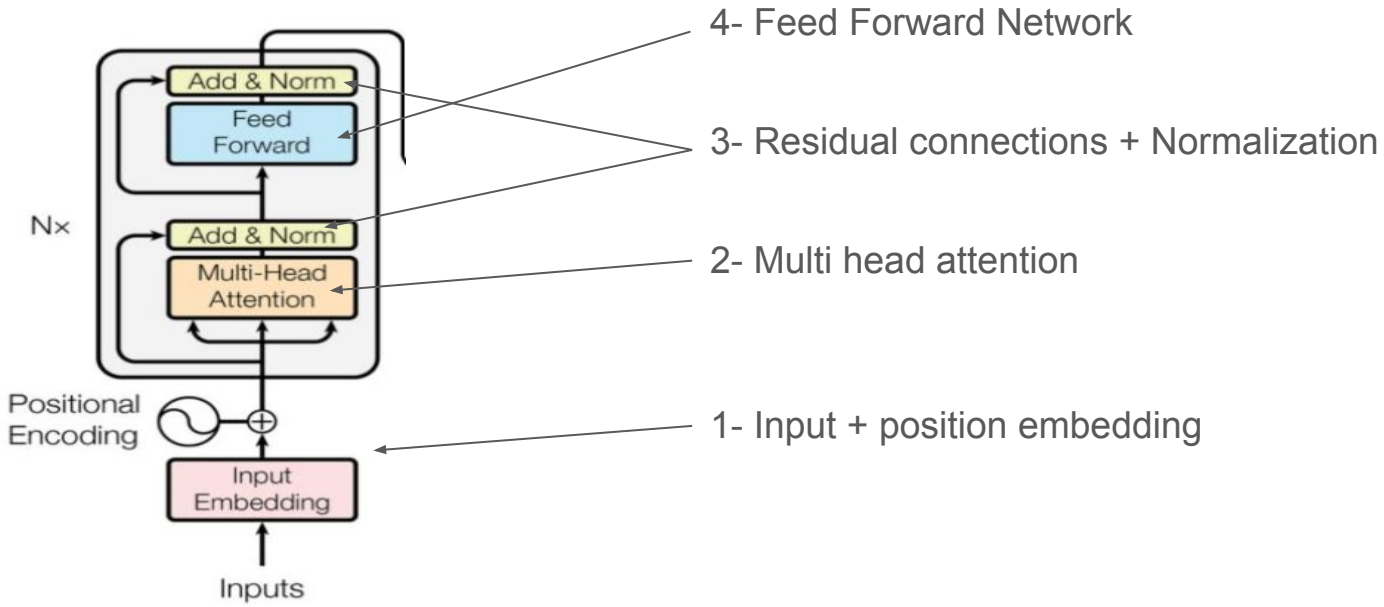
Background

LLM’s are based on the transformer architecture from the paper “Attention is all you need” . The original paper’s transformer was made for text translation and constituted from 2 main parts , an encoder and a decoder.



Background

The Encoder is responsible to find context/semantics/meanings of the prompt and it takes as input tokens with input embeddings which passes through a multihead attention layer then to a feed forward network layer.



Background

1- Input + Position Embeddings:

A- Tokenizations:

The raw text input is transformed into a sequence of tokens where each token has a unique ID and a unique embedding depending on the learned value of this token through a process of tokenization.

All the possible tokens consist the Vocabulary of the Language.

Special Tokens?

Vocabulary examples:

1- Token = word

2- Token = Character

3- Token = subwords

4- Byte-per-encoding(most used): Combines the previous implementations to add also the most frequent pairs i.e. a + n -> an

Background

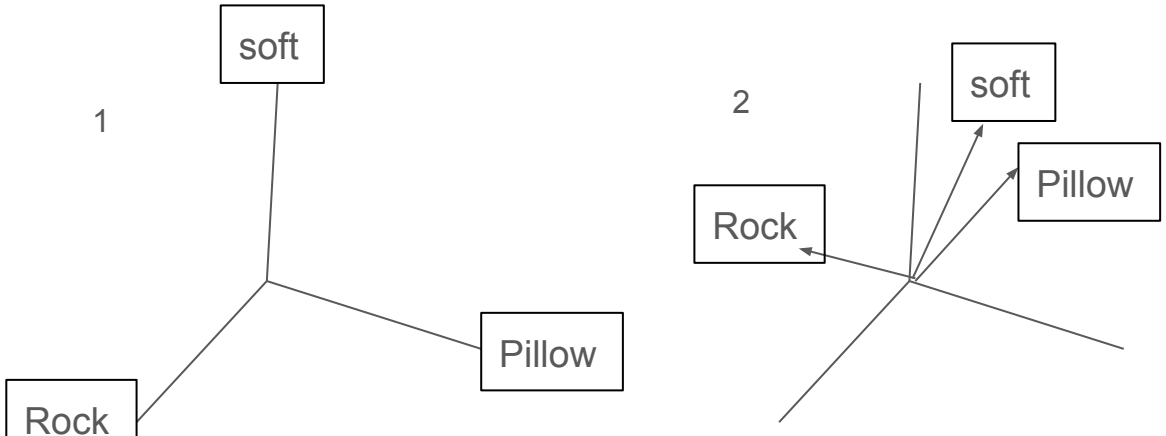
1- Input + Position Embeddings:

B- Embeddings:

Each token now in the vocab needs a representation that reflects the attributes/meanings of this token

To relate all the tokens to each other , each token is represented by a vector which makes comparisons between other tokens easier if they are projected into a vector space in a way that closer tokens sit closer to each other.

- One-hot encoding(Fig 1): assigns each token a binary representation
- Learned embeddings(Fig 2): Learn the embeddings of each token along the training process.



Background

1- Input + Position Embeddings:

C- Position Embeddings:

Before giving the token to the transformer to be processed, additional information needs to be added in order to know the relation between each token which is the position. Since that extra info makes a big difference when computing relations between words/tokens.

- Learned Position-specific embedding : is based on training but suffers when we encounter a sequence longer than what was trained on.

- Hardcoded position-specific embedding: adding a vector to reflect position using sin and cosin functions to relate tokens by making closer indices have a higher frequency then longer ones.

- RoPE: Rotary Position Embeddings : Instead of adding a position vector to the token we rotate the matrices holding the data itself at the level of the multi head attention since this is where the position really matters.

Background

2- Multi Head Attention:

After the complete input with position embeddings are passed to the next section the multi head attention phase starts.
At this phase we compute this input along with the query,key,value matrices in order to find the semantics/context of each token compared to the others.

Example: The leaves of the tree blossom in Spring

Spring could have different meanings depending on the context so in this phase the multi head attention relates it to all the tokens in the sentence to find the suitable one.

The query is responsible to ask the question of what can other tokens help.
The Key is responsible to answer the query with corresponding values.
The Value gives the necessary information.

The multiplication of the query and key will act as a diluter or concentrator to the value depending if there is a relation or not

Background

2- Multi Head Attention:

This operation is carried with multiple heads as each head could relate to different relations(different query , key , values) and is carried multiple times to refine the values more and more.

The key calculation is done as per this equation.

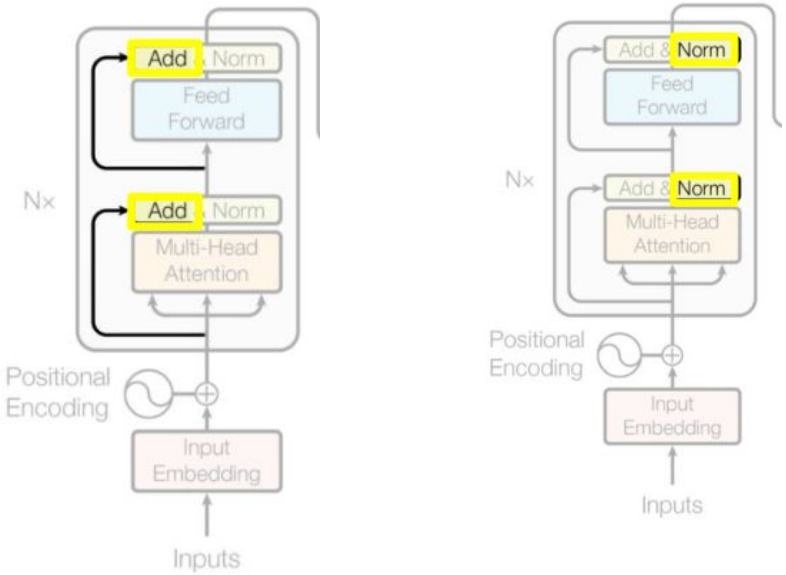
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V,$$

There are different evolutions of this section that will be discussed later.

Background

3- Residual connections + Normalizations:

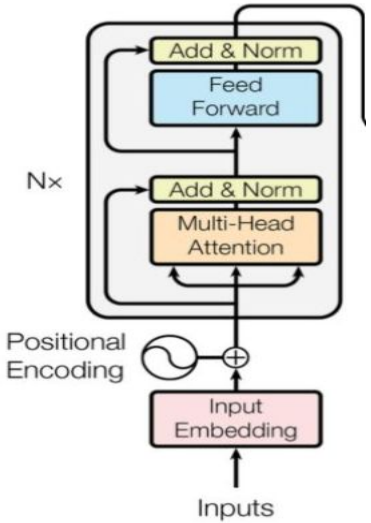
At each section we normalize the results and at the same time we give the original values in addition to the computed ones which enriches the results.



Background

4- Feed Forward Neural Network:

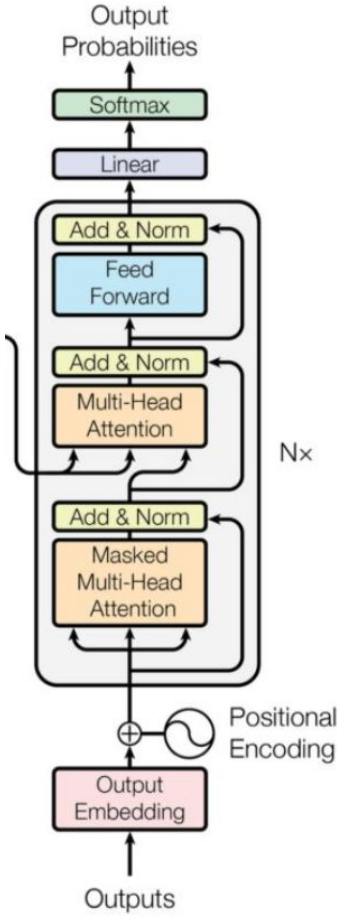
After the attention layer each process will pass through the feed forward network in order to compute non linear relations and enrich the results even more.



Background

Decoder: is the section where the the results of the encoder are used to output token predictions one after the other until the sequence ends.

It has the same section as an encoder but with an additional multi head attention layer that relates the newly outputted tokens with the values computed from the encoder to have some relations to them

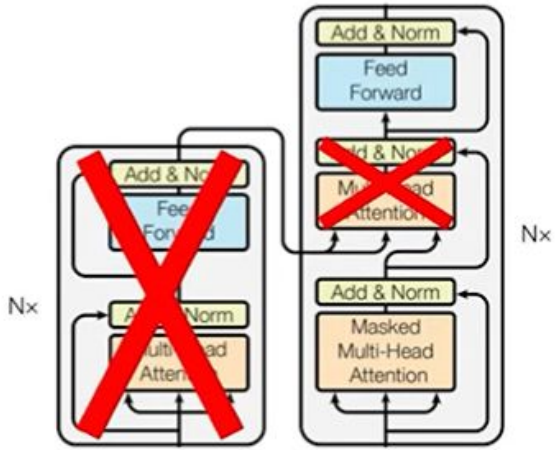


LLM’s

Large Language models are based on the Transformer architecture and take certain aspects of it.

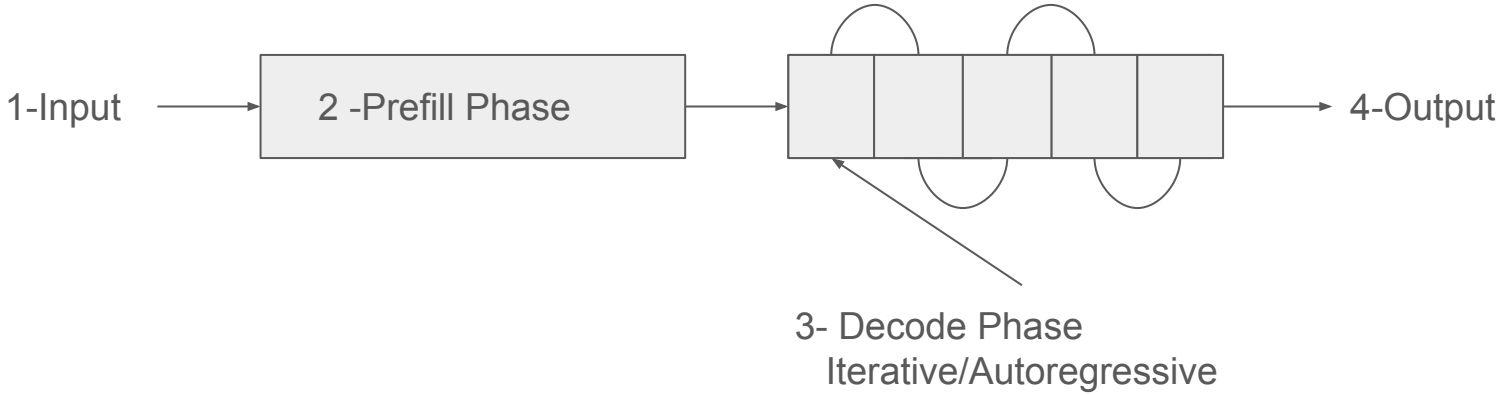
It is constituted of only the decoder part which simplifies the approach and model and it was found that it gives same results as an encoder-decoder model but with less complexity.

As a result any relation between the encoder/decoder is removed and we have only a decoder architecture. When we add an input prompt it is processed in the decoder which will act as an encoder in this case and then the output will be generated autoregressively based on it.



LLM’s

The general process flow of this model is shown in the graph below.



LLM’s

1- Input:

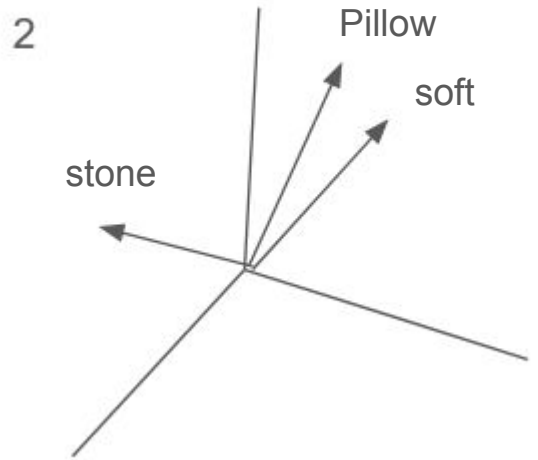
The input in this model basically shares the same aspects of the original transformer model and what differs is usually how words are represented (Vocab) and how positional embedding is represented (learned -, hardcoded , most popular now RoPE)

soft=(0.95 , 0.32 , 0.01)

pillow=(0.32 , 0.95 , 0.01)

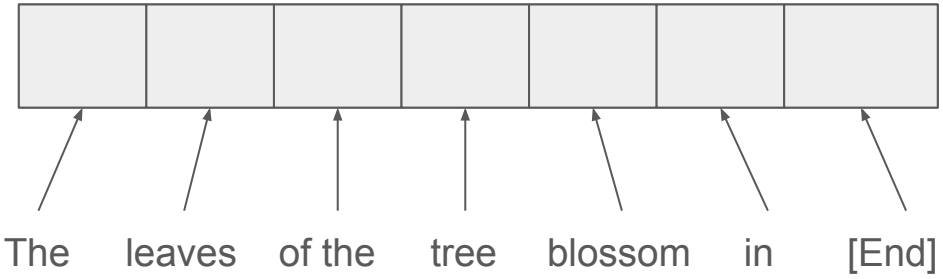
stone=(0.01 , 0.01 , 0.95)

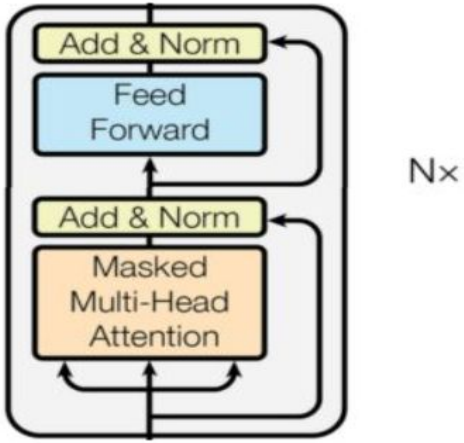
And then rotations will be seen in the matrices in attention.



LLM’s

2- Prefill Phase: Each token will attend to the other along with their position to capture the semantics of the sentence on order to generate responses accordingly using the decoder only model.





This process can be done in parallel since we have all the tokens present .

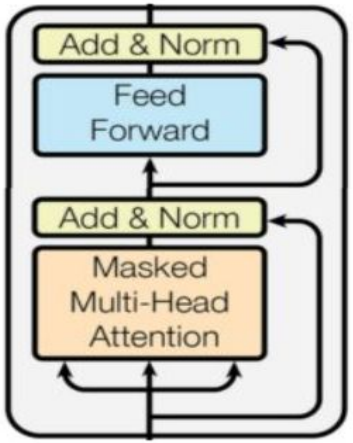
For example we can offload this to multiples GPU’s so that each one can handle a different token and then we merge the results.

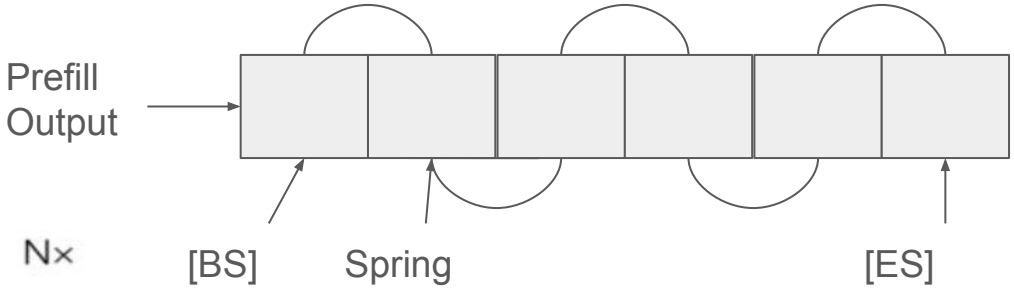
LLM’s

3- Decode Phase:

The output of the prefill phase will be used to start generating the tokens in the decode phase iteratively.

As each token gets generated it uses the previous generated tokens and the context from the input until the output is finished

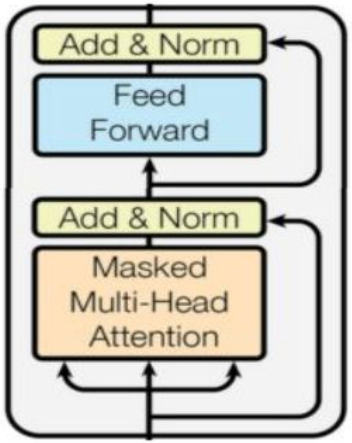


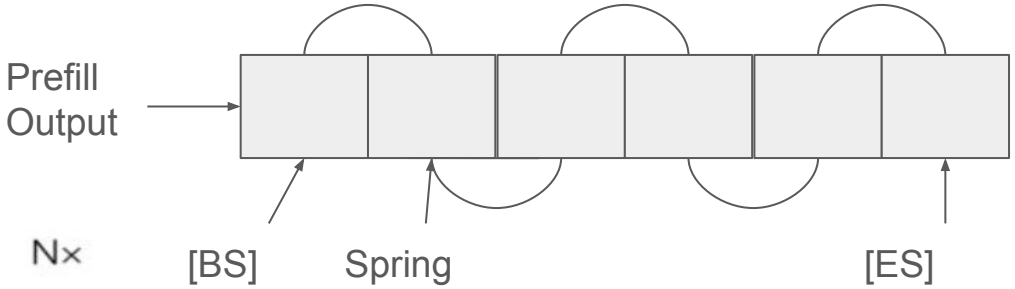


LLM’s

4- Output

The output will ultimately be the generated sequence in the decode phase which would end in a specific token that terminates the request. Many options can lead to the sequence to end : A special hardcoded counter that prevents over generated tokens ,exceeding the context size or the end token has the highest probability once considering new output.





LLM’s

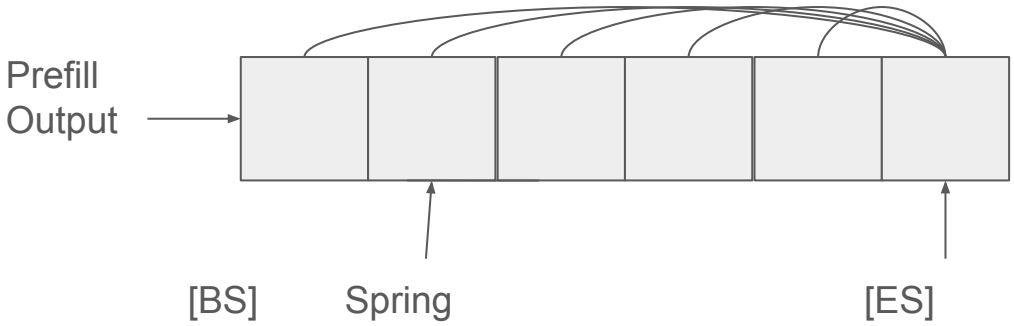
Extra Information:

The sampling choice for output:

-Highest probability / probability distribution.

-Top K / Top P.

After each token gets generated we save the key/value inside a KV cache to not have extra calculations.



Optimizations: Overview

1-Attention optimizations

2-Feed Forward Network Optimizations

3-KV Cache optimizations

4-Scheduling Optimizations

Optimizations: Attention Optimizations

1-Flash attention

2-Sliding Window attention

3-Group Query attention

Optimizations: 1-Flash Attention

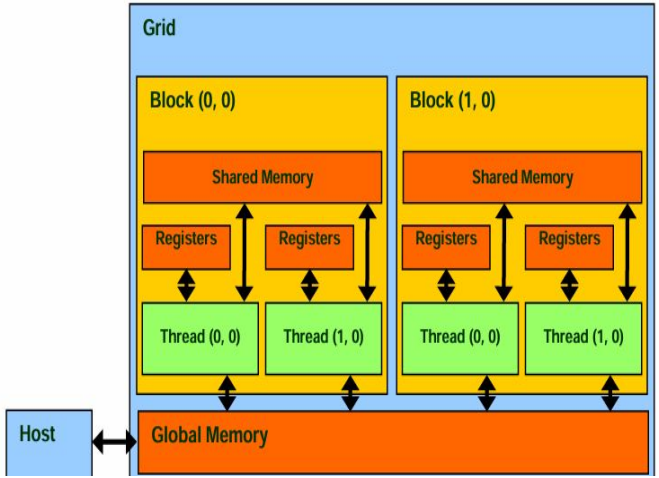
It is basically a modified version of the normal attention in order to compute the attention scores with less and less memory calls inside the GPU.

Originally: we have a lot of copying from the global memory to the shared memory i.e. copy the Q, K matrices to compute attention -> copy again to compute softmax -> copy again to compute the results with V matrix -> A lot of memory calls

Global Memory size: 8 GB to 192 GB

Shared Memory size: 48 KB to 228 KB per SM

Simple Nvidia GPU architecture



Optimizations: 1-Flash Attention

The original method , due to softmax, requires the complete computation of all the matrices in order to be computed.

Optimization: we compute the output iteratively by basically squishing the different computations into one.Compute the softmax of Q.K and then the Dot product with V in one pass. Formula below.

$$\text{softmax}\left((s_1 \quad s_2 \quad s_3)\right)\begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

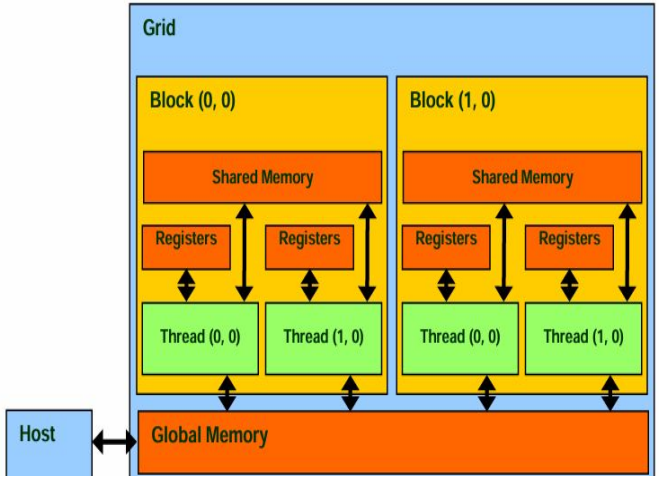
$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

$$m_{t+1} = \max(m_t, s_{t+1}) \qquad \alpha_t = e^{m_t - m_{t+1}}$$

$$N_{t+1} = \alpha_t N_t + e^{s_{t+1} - m_{t+1}} v_{t+1}$$

$$D_{t+1} = \alpha_t D_t + e^{s_{t+1} - m_{t+1}}$$

Simple Nvidia GPU architecture



Optimizations: 1-Flash Attention

The original method , due to softmax, requires the complete computation of all the matrices in order to be computed.

Optimization: This will result in more computations

but ultimately less memory copying which in

reality leads to faster computations overall.

You would load only the q , k , v matrices [N,dmodel]

Instead of loading these matrices separately and

then load their result which is [N,N] which takes

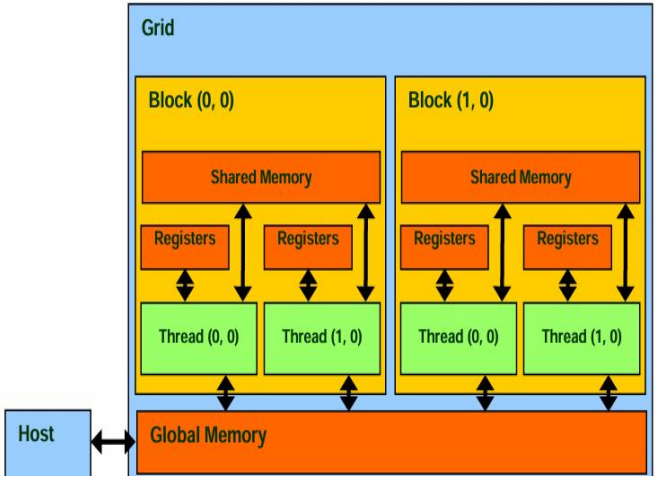
more memory. Example below:

Sequence Length N: 4096 , Dmodel = 64 , 2 bytes/token

Each matrix: N * Model * 2 = 0.5 MB approx.

Q.K = N * N * 2 = 32 MB approx.

Simple Nvidia GPU architecture



Optimizations: 1-Flash Attention

1-Flash Attention: The original method , due to softmax, requires the complete computation of all the matrices in order to be computed.

Variations:

FA1:The K , V matrices are fixed in the outer loop and

then the Q matrix will be inside.

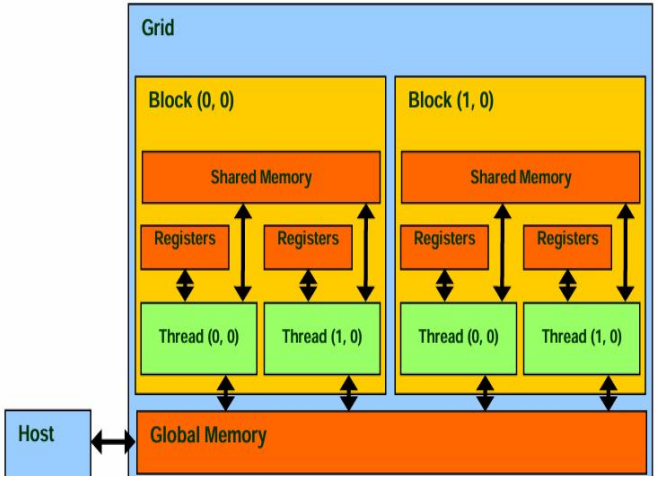
Computes partial output

FA2:Flipped the loops.

FA3:Optimized for the hopper Nvidia serieses

FA4:Optimized for the blackwell nvidia serieses

Simple Nvidia GPU architecture

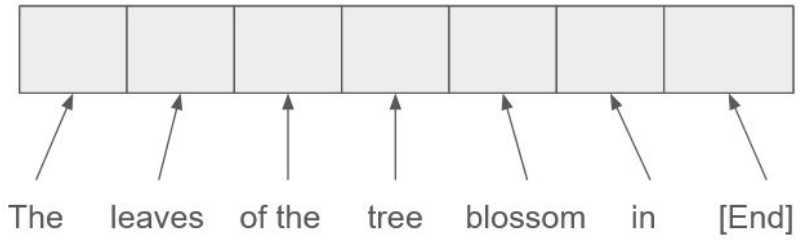


Optimizations: 2-Sliding Window Attention

Instead of letting all the tokens attend to each other we only let the tokens behind a certain window W to interact.

[The] will only attend to itself -> [leaves] will attend to [the] and itself ->etc..

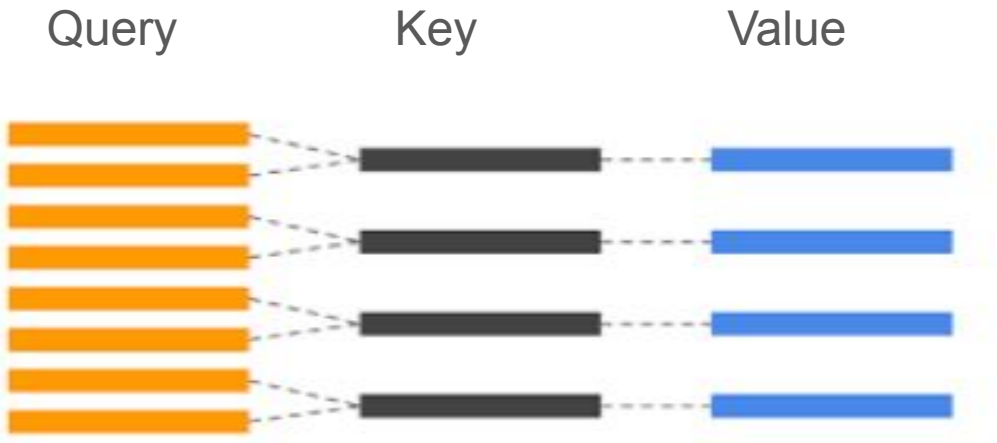
Once blossom interacts with Tree -> tree would have already interacted with the ones before which saves computations.



Optimizations: 3- Group Query Attention

Instead of having all the heads with different Key , Value matrices we share some of them with each other based on a group which is 1<G<H

Each head in a group will have to share the Same Key / Value matrices but will have different query heads. Also saves in KV cache memory footprint.



Optimizations: Feed Forward Network Optimizations

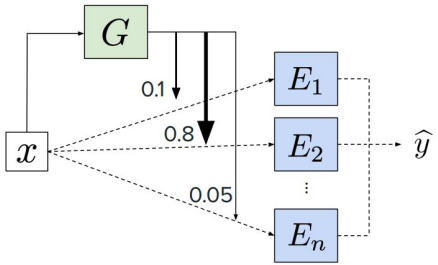
1-Mixture of experts: The Feed Forward network is much denser in terms of parameters compared to the multi head attention layer , therefore if each token needs to be passed to it we would have a lot of computations per token.

Therefore , we introduce multiple experts in it , Which act as different /specialized / smaller Feed Forward networks which would handle the token based on how much it relates to this head.

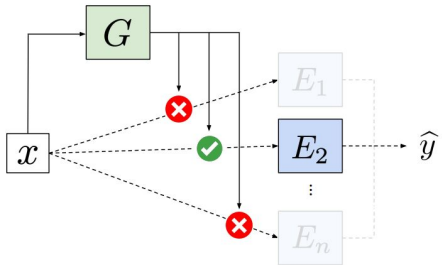
Dense MoE: Which is the weighted average of the token with respect to all the Experts.

Sparse MoE: Which is the weighted average of the token with respect to some K of the Experts.

Dense



Sparse



Optimizations: KV Cache Optimizations

1-Fragmentation (problem rather than optimization)

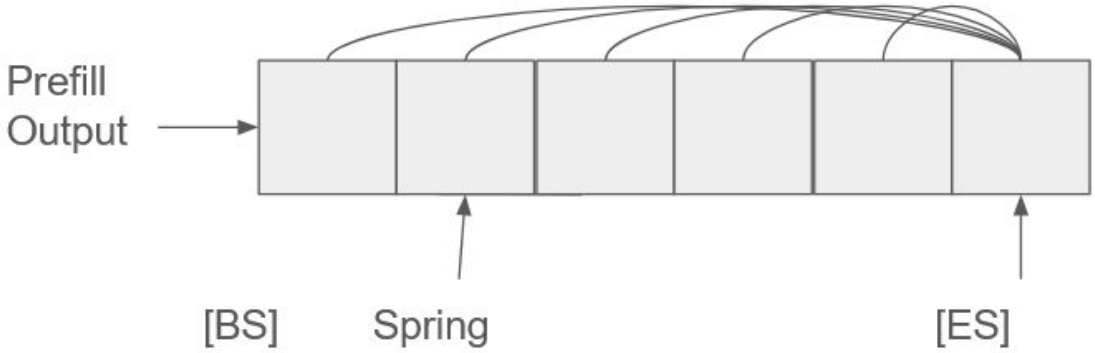
2-Paged Attention

3-Radix attention

Optimizations: 1- Fragmentations

Actually this one is a problem rather than an optimization.

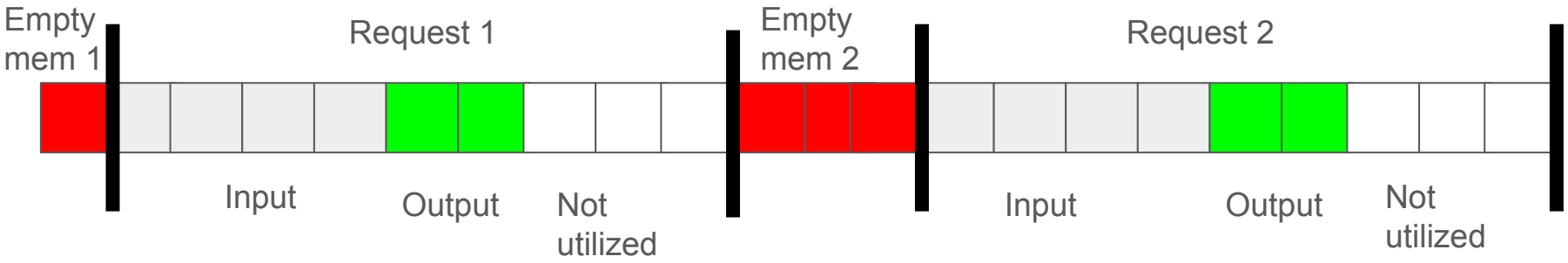
Recall : KV cache is a memory location to save already computed tokens results instead of recomputing it from scratch everytime we output a new token



Optimizations: 1- Fragmentations

1-Fragmentation : Actually this one is a problem rather than an optimization.

Naive Approach : We give each request a fixed size in the memory block (KV cache)



Two Types of fragmentations: Internal and External.

Internal Fragmentation: This happens when we reserve extra blocks for a request which did not use all the storage(white blocks)

External Fragmentation: This happens when the memory get fragmented and new requests could not fit.(red blocks)

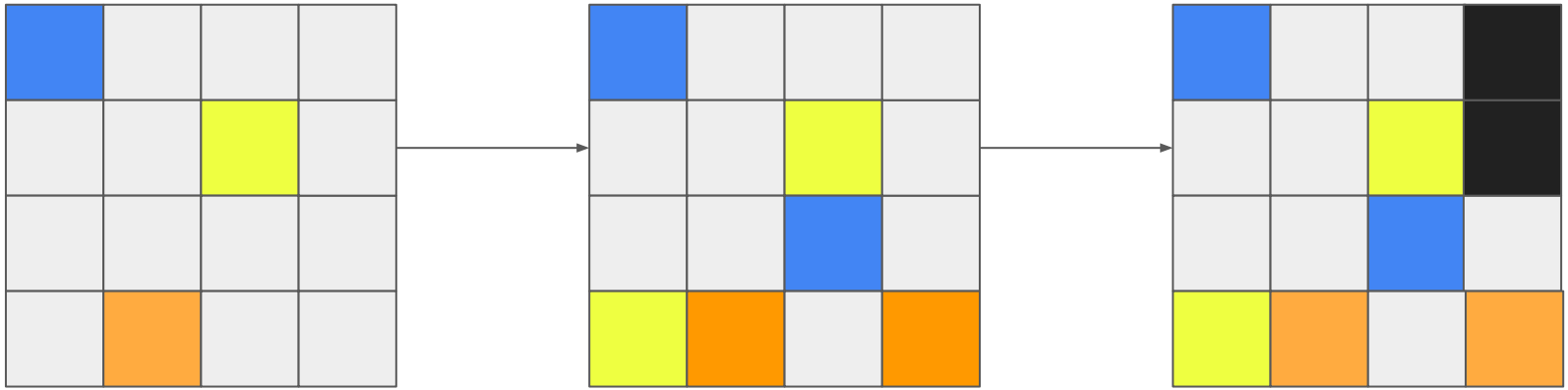
If a new request is issued and wants for example 4 MB worth of blocks but the empty Mem could not allocate a 4MB contiguous block of memory since there are 4MB worth of memory but fragmented

Optimizations: 2- Paged Attention

2- Paged Attention :This implementation tries to solve the problems that we encounter in the naive approach which are fragmentation. It is inspired by virtual memory in OS that uses pages to manage memory.

Basically the memory will be divided into blocks of fixed number of tokens.

Each request will start with 1 block and then add more as needed.Also they can share blocks (No need to be contiguous) . Assume each request has a different color and shared blocks have the color black



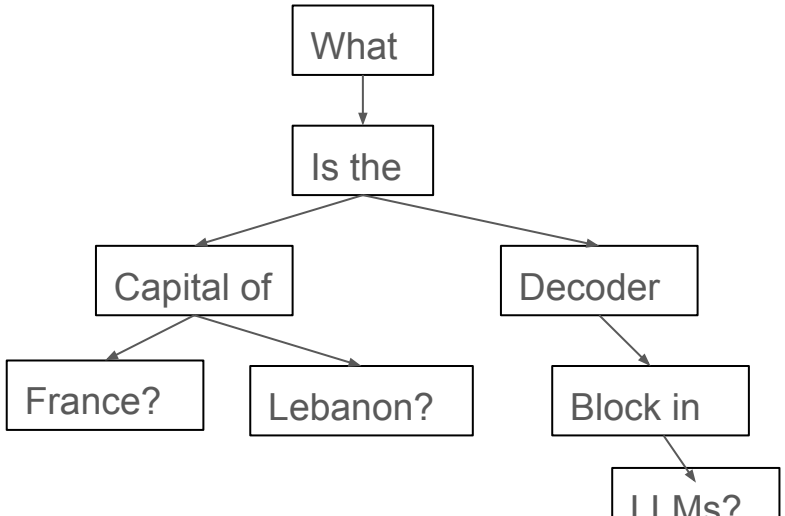
Optimizations: 3- Radix Attention

3- Radix Attention :Takes a similar approach to paged attention but build the memory as a Tree where requests will share some prefixes (nodes \ root) and build the memory accordingly.

Request 1: What is the capital of France?

Request 2: What is the decoder block in LLMs?

Request 3: What is the capital of Lebanon?



Optimizations: Scheduling Optimizations

1- Scheduling policy

2-Batching

3-Sarathi-serve

Optimizations: 1- Scheduling Policy

Basically how can we handle multiple requests to the same model and what is the order of serving?

-First-Come, First-Served (FCFS) : This is a naive approach , but it could suffer if a long request starts which could leave others waiting.

-Shortest Job First (SJF): Another naive approach which is hard to predict since most of the time we do not know how long the response will be

-Priority of requests: Depends on the priority of each request , the highest priority passes first and in case of conflicting priorities we can choose any other scheduling policy to resolve this conflict

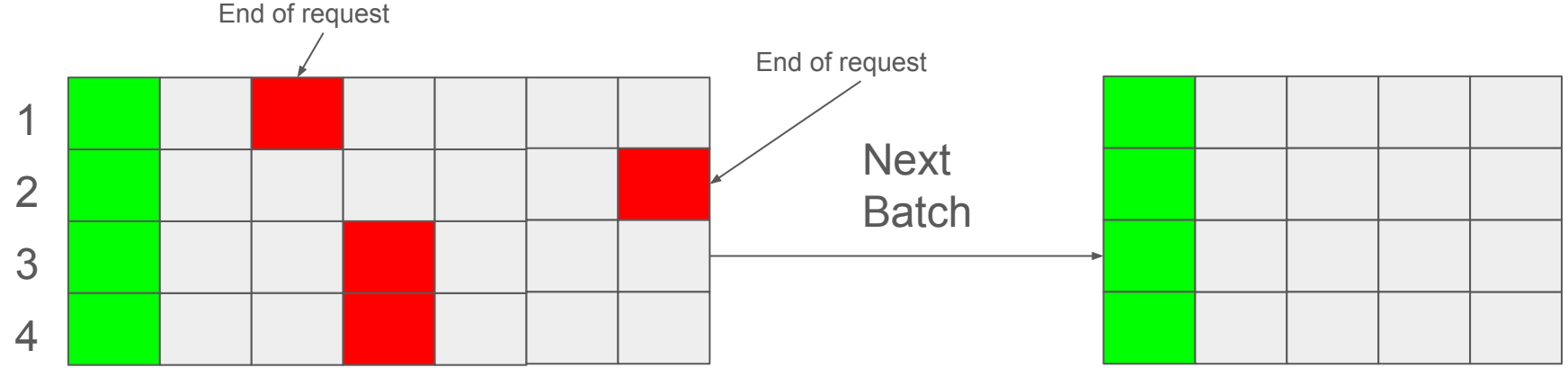
Optimizations: 2- Batching

It is the grouping of multiple requests to be served at the same time .There are multiple grouping metrics/criteria to have in order to choose the requests.

-Static: Have a fixed number of requests per batch . In addition , we wait until all the requests in a batch are finished in order to add new ones.

Example below each row is a request and we assume they are in parallel.

Red means the end of the sequence and Green means the start of a new Request

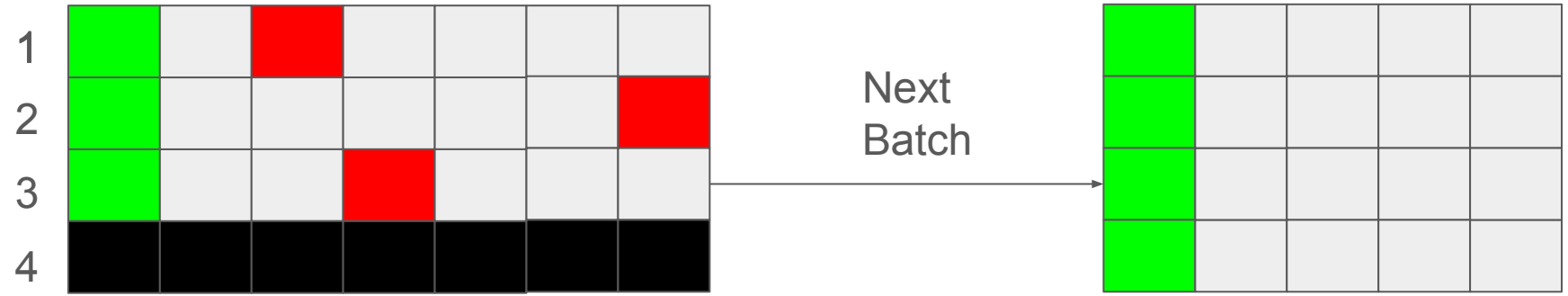


Optimizations: 2- Batching

It is the grouping of multiple requests to be served at the same time .There are multiple grouping metrics/criteria to have in order to choose the requests.

-Dynamic: Have a fixed number of requests per batch plus a timer so that the requests are not delayed by much . In addition , we wait until all the requests in a batch are finished in order to add new ones. Example: below each row is a request and we assume they are in parallel.

Red means the end of the sequence , black row means the request did not arrive on time. Green means the start of a new Request



Optimizations: 2- Batching

It is the grouping of multiple requests to be served at the same time .There are multiple grouping metrics/criteria to have in order to choose the requests.

-Continuous: As soon as a request finishes it is replaced by another one waiting.

Example below each row is a request and we assume they are in parallel.

Red means the end of the sequence , Green means the start of a new Request



Optimizations: 3- Sarathi Serve

Generally , schedulers either prioritises the Prefill Phase (Prompts) or the decode phase(generated tokens / output) of the requests. Usually , the batch would ,depending on the priority , set batches with only prompts or outputs.

In case we have a long prompt , the output of other requests will be stalled until it is finished and inversely if we have a batch of output the prompts execution would also be delayed .

Sarathi serve , solve this by having a hybrid batch ,usually with a fixed token cap, that combines chunks of a long prompt with the output of other requests to allow a consistent flow in requests.